

App Store & HealthKit Compliance Audit

DiabetesHbA1cPrediction iOS Application

Audit Date:	March 04, 2026
Scope:	46 Swift source files, Info.plist, entitlements, project configuration
Frameworks:	HealthKit, CoreData, SwiftUI
App Purpose:	Diabetes management with HbA1c prediction using glucose, meal, and exercise data

Executive Summary

Severity	Count	Key Issues
[CRITICAL]	1	Invalid iOS deployment target (26.0)
[HIGH]	2	HealthKit write permission mismatch; extensive console logging of health data
[MEDIUM]	4	Missing medical disclaimer in main UI; no CoreData encryption; health data in share export; fatalError in production code
[LOW]	2	Realistic PII in preview/test data; glucose unit conversion lacks tests

The app has a sound architecture with no network calls, no third-party analytics, and no advertising frameworks. All health data stays on-device. However, several configuration and compliance issues must be resolved before App Store submission.

Detailed Findings

[CRITICAL] Invalid iOS Deployment Target

Location:	project.pbxproj (6 occurrences across all build configurations)
Issue:	IPHONEOS_DEPLOYMENT_TARGET is set to 26.0 across all Debug and Release configurations. iOS 26.0 does not exist. The current latest is iOS 18.
Details:	Found in all 6 build setting blocks in the Xcode project file. This will prevent archiving and App Store submission.
Impact:	App cannot be built for a real device or submitted to App Store Connect. Xcode will reject the configuration.
Recommendation:	Set the deployment target to a valid iOS version (e.g., 16.0 or 17.0). Update in Xcode under each target's General tab or directly in project.pbxproj.

[HIGH] HealthKit Write Permission Declaration Without Implementation

Location:	Info.plist (line 23-24) and project.pbxproj INFOPLIST_KEY settings; HealthKitManager.swift (line 82)
Issue:	Info.plist declares NSHealthUpdateUsageDescription ("This app writes exercise and meal data to Apple Health...") but the code requests zero write permissions: toShare is an empty Set.
Details:	The usage description string also appears in the project.pbxproj INFOPLIST_KEY_NSHealthUpdateUsageDescription for both Debug and Release. The HealthKitManager never calls healthStore.save() anywhere in the codebase. The declaration is misleading.
Impact:	Apple reviewers may reject the app for declaring a write permission it never uses, or users may be confused by the authorization prompt. This violates App Store Review Guideline 5.1.1 (Data Collection and Storage).
Recommendation:	Either (a) remove NSHealthUpdateUsageDescription from Info.plist and both INFOPLIST_KEY entries if write is not needed, or (b) implement actual HealthKit write operations and populate the toShare parameter with the relevant HKSampleTypes.

[HIGH] Extensive Console Logging of Health-Related Operations

Location:	40+ print() statements across HealthKitManager.swift (11), HbA1cPredictionEngine.swift (13), GlucoseLogView.swift (3), ExerciseLogView.swift (3), MealLogView.swift (2), PersistenceController.swift (1), and others
Issue:	Health-related error messages and operational status are logged to the system console via print() without any conditional compilation guards.
Details:	Examples: "Error fetching glucose readings: ...", "Successfully saved prediction result to Core Data", "Error fetching user demographics: ...". These appear in device console logs that can be read via Xcode, Console.app, or crash reports.
Impact:	Console logs may expose sensitive health operation details. While error messages themselves do not contain raw health values, they reveal the types of health data the user has and operational patterns. This is a privacy concern under GDPR and best-practice guidelines for health apps.
Recommendation:	Wrap all print() calls in #if DEBUG blocks, or replace with os_log() using appropriate privacy levels. For example: os_log("Glucose fetch error", log: .default, type: .error)

[MEDIUM] Medical Disclaimer Only Visible in Share/Export Text

Location:	PredictionView.swift (line 232)
Issue:	The disclaimer "This prediction is for informational purposes only and should not replace professional medical advice" appears only when the user exports/shares a prediction. It is not displayed in the Dashboard, PredictionView, or any primary UI.
Details:	The app generates HbA1c predictions with risk categories and clinical recommendations. Users see these predictions prominently in the Dashboard and Prediction views without any accompanying disclaimer.
Impact:	App Store Review Guideline 1.4.1 requires health apps to clearly indicate they do not provide medical advice. Users may make health decisions based on predictions without understanding their limitations.
Recommendation:	Add a visible disclaimer footer or banner to the DashboardView and PredictionView wherever predictions or risk categories are displayed. Consider also adding a disclaimer during onboarding.

[MEDIUM] No Data Protection on CoreData Storage

Location:	PersistenceController.swift (lines 156-172)
Issue:	The CoreData persistent store is loaded with default settings and no NSFileProtection attributes are configured.
Details:	container.loadPersistentStores() uses default options. No NSPersistentStoreDescription options are set for NSPersistentStoreFileProtectionKey. The SQLite database file on disk is not encrypted at rest beyond device-level encryption.
Impact:	If a device is compromised (jailbreak, forensic extraction), health data including glucose readings, meal logs, and HbA1c predictions could be read from the unprotected database file.
Recommendation:	Set NSFileProtection on the store description: description.setOption(FileProtectionType.completeUntilFirstUserAuthentication as NSObject, forKey: NSPersistentStoreFileProtectionKey) before loading.

[MEDIUM] Health Data Shared Without User Warning via ShareSheet

Location:	PredictionView.swift (lines 204-236)
Issue:	The prepareShareContent() function builds a detailed text summary including predicted HbA1c value, confidence level, risk category, and contributing factor values, then passes it to the system ShareSheet.
Details:	The share content includes specific health metrics. Once shared via Messages, email, or social media, there is no way to retract it. No confirmation dialog warns the user that they are about to share personal health data.
Impact:	Sensitive health information could be inadvertently shared publicly, creating privacy and potential legal exposure.
Recommendation:	Add a confirmation alert before presenting the ShareSheet that explicitly states the user is about to share personal health information. Consider offering a summary-only option that omits detailed contributing factor values.

[MEDIUM] fatalError() Calls in Production Code Path

Location:	PersistenceController.swift (lines 144, 170)
Issue:	Two fatalError() calls exist in code that runs in production: one in preview data seeding and one in the main persistent store loading: fatalError("Core Data store failed to load: ...")
Details:	While the preview data fatalError may only trigger in SwiftUI previews, the store-loading fatalError on line 170 is in the production code path. If CoreData fails to load (e.g., migration issue, disk full), the app will crash immediately.
Impact:	App Store reviewers may flag immediate crashes. Users who encounter CoreData migration issues after an update will lose access to the app entirely.
Recommendation:	Replace fatalError with graceful error handling: show an alert to the user, attempt recovery, or present a read-only fallback state.

[LOW] Preview/Test Data Contains Realistic Health Profiles

Location:	PreviewData.swift (lines 54-94), PersistenceController.swift (lines 52-94)
Issue:	Test data includes a specific health profile (age 52, male, 175 cm, 82 kg, Type 2 Diabetes, heart disease) with realistic glucose values (95-172 mg/dL) and named meals.
Impact:	While preview data should not ship in production builds, if it does, it creates a realistic-looking health profile that could confuse testers or reviewers.
Recommendation:	Use clearly fictional/placeholder data (e.g., age 0, name 'Test User') and ensure preview code is excluded from Release builds.

[LOW] Glucose Unit Conversion Without Unit Tests

Location:	HealthKitManager.swift (lines 332-337)
Issue:	Blood glucose values are converted from mmol/L to mg/dL using a multiplication factor of 18.0. The HealthKit unit construction is complex (moleUnit with molarMass divided by liter).
Impact:	An error in this conversion could cause incorrect glucose values to be stored and used in HbA1c predictions, potentially leading to incorrect health guidance.
Recommendation:	Add unit tests in DiabetesHbA1cPredictionTests to verify glucose conversion accuracy across a range of known values.

Compliance Assessment Matrix

Apple HealthKit Guidelines (Section 27)

Guideline	Status	Notes
27.1 Data Access & Authorization	PASS	Proper HKHealthStore.isHealthDataAvailable() check; correct async authorization flow
27.2 No Third-Party Data Sharing	PASS	No network calls; no external SDKs; all data stays on-device
27.3 No Use for Advertising	PASS	No advertising or analytics frameworks detected
27.4 Encrypted/Protected Storage	FAIL	CoreData store has no NSFileProtection configured
27.5 Device Availability Check	PASS	HKHealthStore.isHealthDataAvailable() called before authorization
27.6 No Clinical Records	PASS	No HKClinicalType or health-records entitlement used

App Store Review Guidelines

Check	Status	Notes
Valid Deployment Target	FAIL	iOS 26.0 is invalid; must be corrected
No Private/Undocumented APIs	PASS	Only public Apple frameworks used
No Hardcoded Credentials	PASS	No API keys, secrets, or tokens found
No External Network Calls	PASS	No URLSession, Alamofire, or HTTP requests
No Deprecated APIs	PASS	All APIs are current
No Tracking Frameworks	PASS	No analytics, ads, or tracking SDKs
Privacy Declarations Accurate	FAIL	NSHealthUpdateUsageDescription declared but not used
Medical Disclaimer (1.4.1)	PARTIAL	Present in share text only; not in main UI
Crash-Free Launch (2.1)	PARTIAL	fatalError on CoreData load failure could crash on launch

Privacy & Data Protection

Check	Status	Notes
No Off-Device Data Transmission	PASS	All data remains local on device
CoreData File Protection	FAIL	No NSFileProtection configured
Minimal Permission Footprint	PASS	Only HealthKit permissions; no camera/location/contacts
No Health Data in Logs	FAIL	40+ unguarded print() statements in production code
Share Export Safety	PARTIAL	Exports detailed health metrics without confirmation warning

Remediation Priority

Priority 1 - Must Fix Before App Store Submission

- Fix iOS deployment target from 26.0 to a valid version (e.g., 16.0 or 17.0)
- Remove NSHealthUpdateUsageDescription from Info.plist and project.pbxproj, or implement HealthKit write functionality

Priority 2 - Should Fix Before Production Release

- Wrap all print() statements in #if DEBUG or replace with os_log() with appropriate privacy levels
- Add NSFileProtection to CoreData persistent store description
- Add visible medical disclaimer to Dashboard and Prediction views
- Replace fatalError() in PersistenceController.swift with graceful error handling

Priority 3 - Recommended Improvements

- Add confirmation dialog before sharing health data via ShareSheet
- Replace realistic PII in preview/test data with clearly fictional placeholders
- Add unit tests for glucose mmol/L to mg/dL conversion
- Consider adding a data privacy policy accessible within the app

Positive Findings

- ✓ No network calls or external data sharing detected - all health data stays on-device
- ✓ No advertising, analytics, or tracking frameworks integrated
- ✓ No private or undocumented Apple APIs used
- ✓ No hardcoded API keys, secrets, or credentials found
- ✓ HealthKit authorization follows Apple's recommended async/await pattern
- ✓ Proper HKHealthStore.isHealthDataAvailable() device check before authorization
- ✓ No clinical health records (HKClinicalType) accessed
- ✓ Minimal permission footprint - only HealthKit, no camera/location/contacts/etc.
- ✓ Clean, well-documented Swift codebase with proper separation of concerns
- ✓ CoreData deduplication logic prevents duplicate HealthKit imports